

Data Structure

Level 1

Doubly Linked List Operation

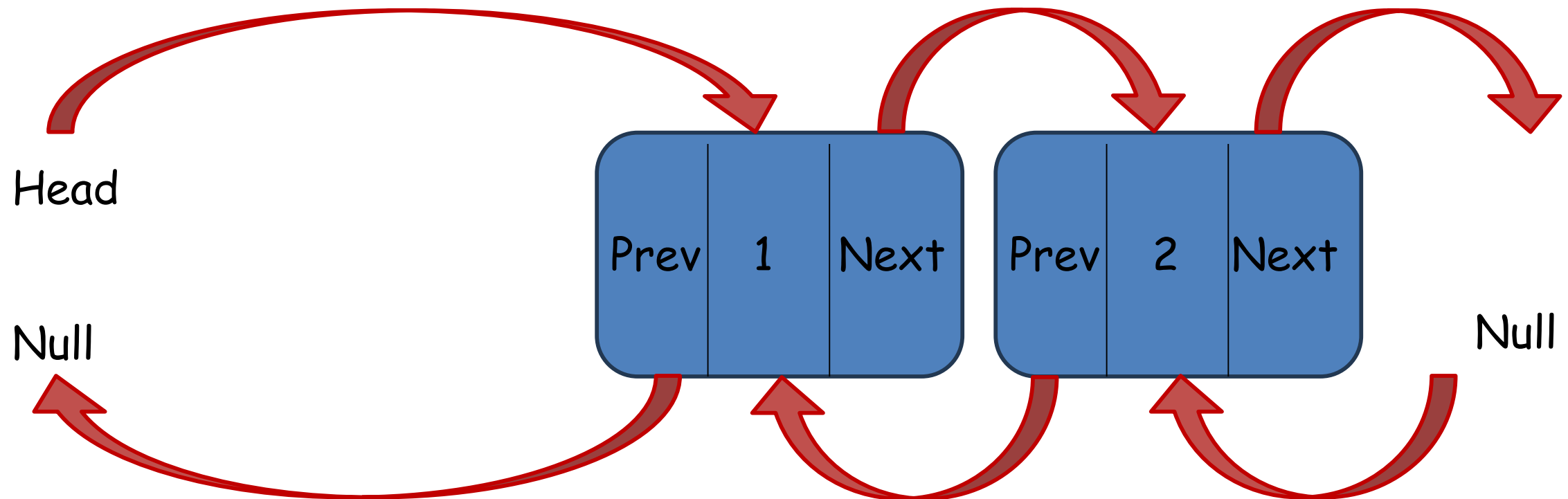
The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, adding a human element to the technical theme.

Data Structure

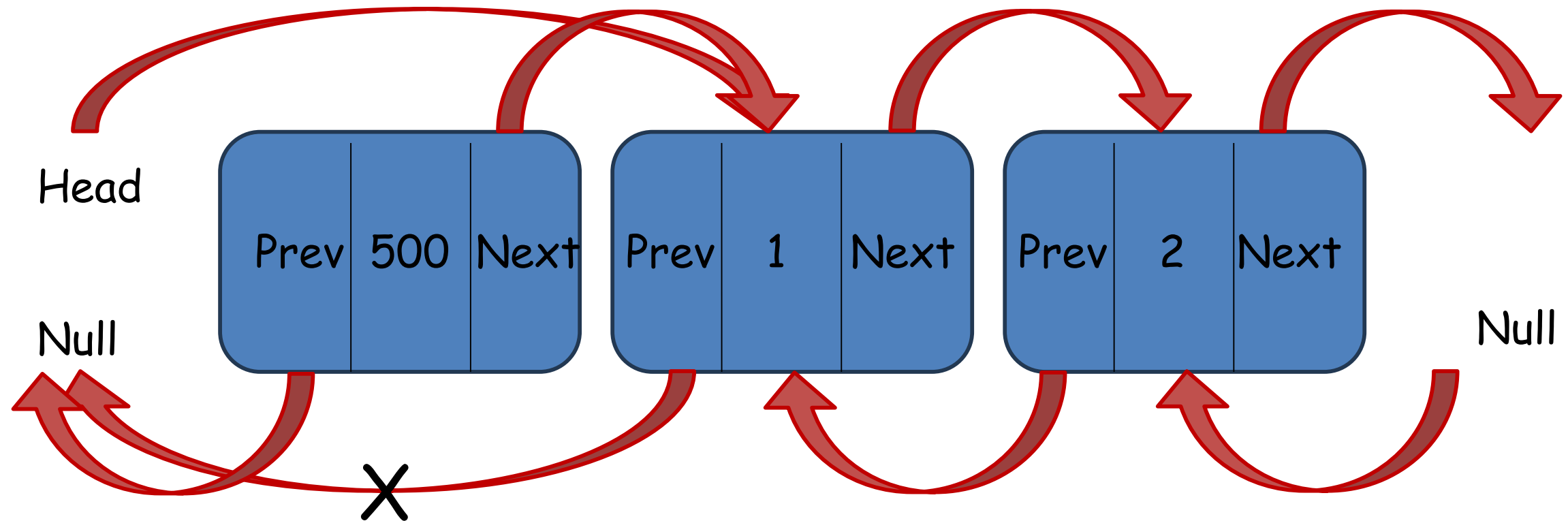
Level 1

Insert At Beginning

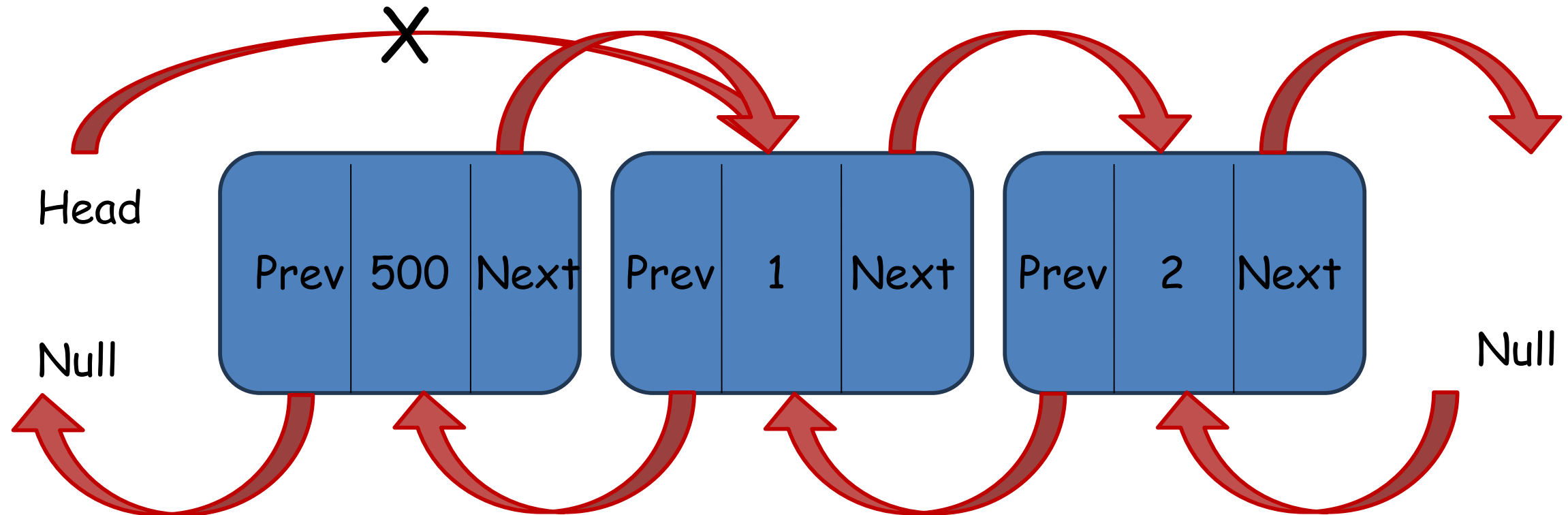
Doubley Linked List Data Structure



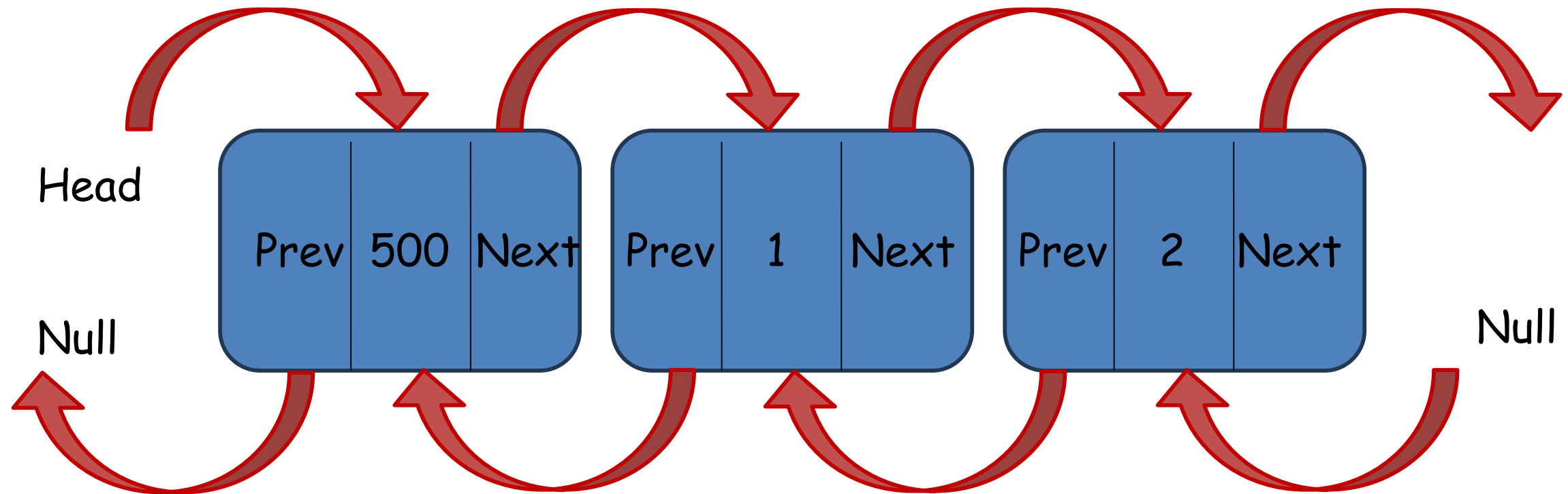
Doubley Linked List Data Structure



Doubley Linked List Data Structure



Doubley Linked List Data Structure



🔍 Definition

Doubly Linked List (DLL):

A list where each node contains:

- Components

- value → stores the data

- next → pointer to the next node

- prev → pointer to the previous node

This structure allows traversal forward and backward.

Explanation

- ◆ `InsertAtBeginning` ensures bidirectional links are maintained.
- ◆ `PrintNodeDetails` prints each node's previous and next connections.
- ◆ `PrintList` gives a clear overview of the list from head to tail.

□ Procedures (Insert at Beginning)

① Create Node Class: value, next, prev.

② Insert at Beginning:

- Create new node
- Set next of new node to current head
- Update prev of current head (if not NULL) to new node
- Move head pointer to new node

③ Print Node Details: Shows prev, value, next of each node

④ Print Full List: Displays entire list connections with arrows

⑤ Main:

- Initialize head
- Insert nodes 1 → 5
- Print list and details

! Example

After inserting nodes 1 $\rightarrow$ 5 at the beginning:

$\rightarrow$ $\square$ Resulting

Linked List Content:

NULL $\leftarrow$ 1 $\rightarrow$ $\leftarrow$ 2 $\rightarrow$ $\leftarrow$ 3 $\rightarrow$ $\leftarrow$ 4 $\rightarrow$ $\leftarrow$ 5 $\rightarrow$ NULL

Linked List Details:

NULL $\leftarrow$ 1 $\rightarrow$ 2

1 $\leftarrow$ 2 $\rightarrow$ 3

2 $\leftarrow$ 3 $\rightarrow$ 4

3 $\leftarrow$ 4 $\rightarrow$ 5

4 $\leftarrow$ 5 $\rightarrow$ NULL

Note

Head's prev is always NULL.
Last node's next is always NULL.

⚖️ Comparison

□ Singly Linked List

- ✓□ Each node has 1 pointer (next)
- ✓□ Traversal only forward
- ✓□ Less memory
- ✓□ Harder to delete specific nodes without previous pointer

□ Doubly Linked List

- ✓□ Each node has 2 pointers (next and prev)
- ✓□ Traversal forward and backward
- ✓□ More memory usage
- ✓□ Easier insertion and deletion anywhere in the list

⊘ Warning

Memory allocated using `new` is not deleted in this program → risk of memory leaks.
Always use `delete` after nodes are no longer needed..

✦✦ Summary

- ✓ Nodes store value, next, prev
- ✓ Supports insertion at the beginning
- ✓ Can print full node connections
- ✓ Bidirectional traversal increases flexibility
- ⊘ Requires careful memory management

Thank you 😊